

HOOPS AI: Transfer a Face Selection

Select a connected region on the source part and highlight a candidate region on the target. Source and target regions may contain different numbers of faces.

`RegionTransfer` handles region matching between two loaded parts. Results are approximate geometric candidates, not verified manufacturing correspondences.

```
In [1]: import hoops_ai
import os
import sys

license_key = os.environ.get("HOOPS_AI_LICENSE")
if not license_key:
    sys.exit("HOOPS_AI_LICENSE environment variable is required.")

hoops_ai.set_license(license_key, validate=True)
```

HOOPS AI

Platform : Windows 11
Architecture : AMD64
Python : 3.12.10

Core	: hoops-ai	1.2.0	
CAD Access	: hoops-exchange	26.8.0.dev1	(build: 87762679 2026-09-09T10:20:34Z)
Conversion	: hoops-converter	26.1.2.dev10	(build: 71b6907 2026-07-24T12:00:12Z)
Insights	: hoops-web-viewer	26.1.4.dev1	(build: 6d78319 2026-09-25T16:18:16Z)

=====
[OK] HOOPS AI License: Valid
=====

1. Load the Parts and Initialize Region Transfer

Run from the `embeddings_pipeline` directory with the HOOPS AI environment and `HOOPS_AI_LICENSE` set. Run once for this pair of files. `RegionPart.load_from_file` encodes one body of a CAD file; pass `body_index` to pick another body of a multi-body file. `RegionTransfer` prepares and aligns both parts once for all later transfers.

```
In [2]: import pathlib
```

```
from hoops_ai.dataset.region_transfer import Region, RegionPart, RegionTransfer

cad_folder = (pathlib.Path.cwd().parent / "packages" / "cadfiles" / "workflow_trans
cad_filename = cad_folder / "nist_mtc_crada_plate_rev-A.SLDPRT"
cad_filename_target = cad_folder / "nist_stc_09_asme1_ap242-e3.stp"

source_part = RegionPart.load_from_file(cad_filename, body_index=0)
target_part = RegionPart.load_from_file(cad_filename_target, body_index=0)
region_transfer = RegionTransfer(source_part, target_part)
```

2. Open the Source and Target Views

The viewers show the whole file, so use files with a single body.

```
In [3]: from hoops_ai.insights import CompareViews

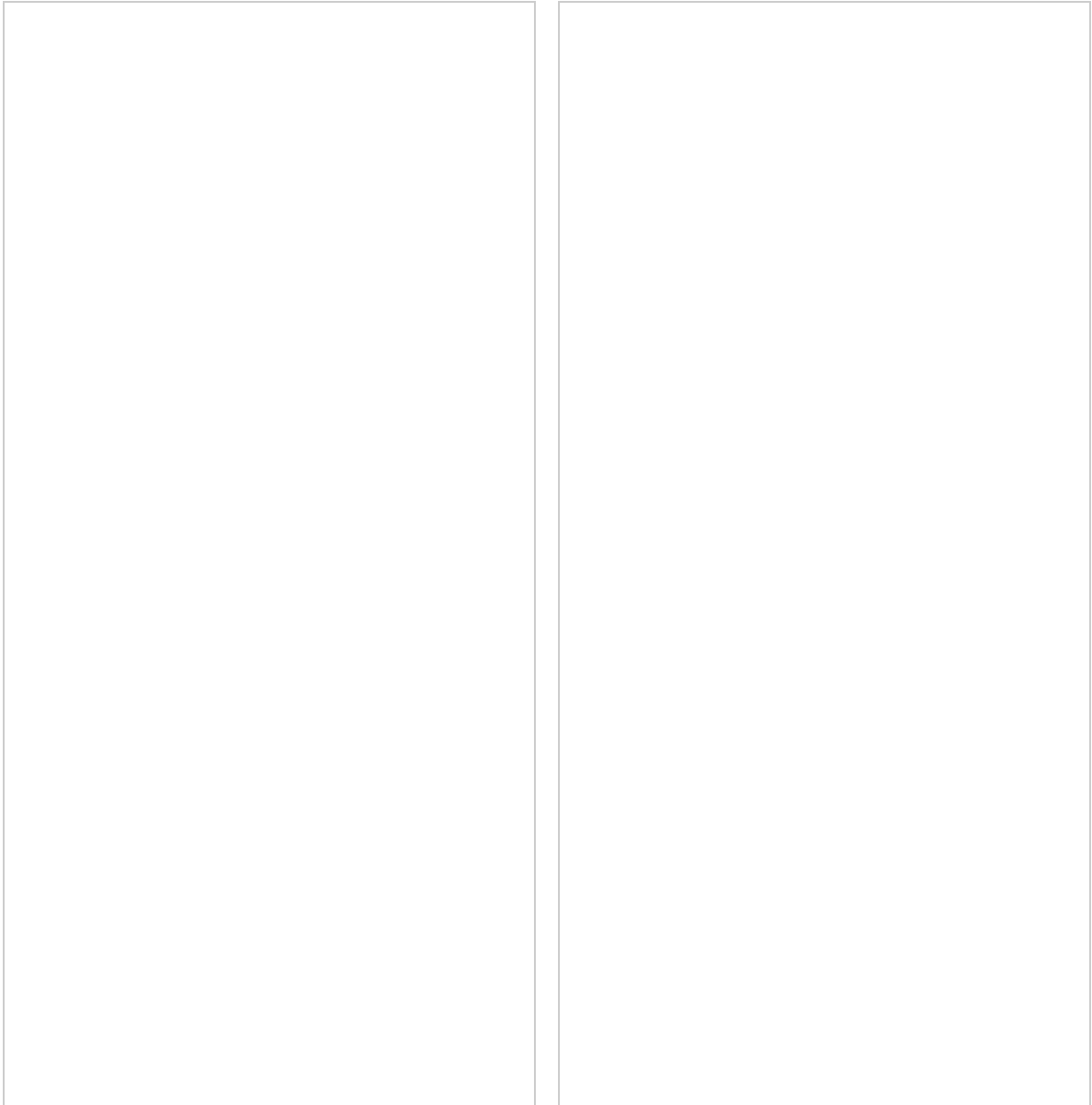
output_dir = (pathlib.Path.cwd() / "out").resolve()

if globals().get("views") is not None:
    views.terminate()
views = CompareViews.start_viewers(cad_filename, cad_filename_target, static_folder

In [4]: views.show()
```

Source: nist_mtc_crada_plate_rev-A.SLDPRT

Target: nist_stc_09_asme1_ap242-e3.stp



3. Select, Transfer, and Highlight

Select one or more face regions in the source viewer, then run the next cell. Connected faces are treated as one region; disconnected faces start separate regions. Each source region and its assigned target match receive the same distinct color.

```
In [5]: source_faces = views.left.get_selected_faces()

if not source_faces:
    print("Select one or more source faces, then rerun this cell.")
else:
    # One Region per group of connected faces; disconnected faces become separate regions
    source_regions = list[Region] = region_transfer.split_selection(source_faces)
    multi_result = region_transfer.map_regions(source_regions)
```

```
# Only matched pairs are colored, so every color appears on both parts.
views.highlight_regions(multi_result.source_regions, multi_result.target_region)
for index, region in enumerate(multi_result.regions, start=1):
    print(
        f"Region {index}: source {list(region.source_faces)} -> "
        f"target {list(region.target_faces)} if region.matched else 'no match (n"
    )
```

Select one or more source faces, then rerun this cell.

Inspect the Result (Optional)

The selected faces are split into connected components. Each component is matched using surface types, internal adjacency angles, normalized local geometry, and surrounding feature context. Smooth same-type neighbors across near-zero-angle boundaries may be grouped to tolerate face subdivisions.

Target assignment is global across the selected source regions: it first maximizes the number of matched components, then minimizes their combined ranking score without allowing target regions to overlap. An unmatched source region remains in the result with an empty target list.

Each component uses the configured topology and geometric search limits independently. Lower discrepancy scores indicate closer agreement, not confidence. Search limits, approximate surface sampling, and ambiguity still require review.

```
In [6]: if source_faces:
    print(f"Detected regions: {len(multi_result.regions)}")
    print(f"Matched regions: {multi_result.matched_count}")
    print(f"All searches complete: {multi_result.complete}")
    for index, region in enumerate(multi_result.regions, start=1):
        print(
            f"Region {index}: source={list(region.source_faces)}, "
            f"target={list(region.target_faces)}, ambiguous={region.result.ambiguous} "
            f"search={region.result.termination.value}"
        )
```

4. Propose and Map Source Regions Automatically

Find up to 40 whole mechanical features in the source part, such as holes, pockets, slots, text and the outer border. All features are mapped together so that each source feature gets a different target region. A distinctive feature with one clear match acts as an anchor: its nearest source features go to the nearest look-alike target features, in the same distance order. Repeated features keep their relative layout. A feature without a good counterpart stays unmatched rather than taking a wrong target. Regions are shown one at a time: the source feature first, then its target in the same color.

```
In [7]: proposals = region_transfer.propose_regions(source_part, limit=40)
multi_result = region_transfer.map_regions(proposals)
views.highlight_regions(multi_result.source_regions, multi_result.target_regions)

for rank, (proposal, region) in enumerate(zip(proposals, multi_result.regions, stri
print(
    f"{rank}: source={list(region.source_faces)} -> "
    f"target={list(region.target_faces)} if region.matched else 'no match (not c
    f"interest={proposal.interest_score:.3f}"
)
print(f"Colored {multi_result.matched_count} of {len(proposals)} source features.")
```

```
C:\Users\LuisSalazar.LY-LS-LEGION\AppData\Local\Temp\ipykernel_44344\1750804783.py:
2: UserWarning: Source region 7: approximate transfer stopped with beam_limit; the r
returned region is the best candidate found, not a proven optimum.
    multi_result = region_transfer.map_regions(proposals)
C:\Users\LuisSalazar.LY-LS-LEGION\AppData\Local\Temp\ipykernel_44344\1750804783.py:
2: UserWarning: Source region 7: no non-overlapping target assignment was available.
    multi_result = region_transfer.map_regions(proposals)
C:\Users\LuisSalazar.LY-LS-LEGION\AppData\Local\Temp\ipykernel_44344\1750804783.py:
2: UserWarning: Source region 8: approximate transfer stopped with beam_limit; the r
returned region is the best candidate found, not a proven optimum.
    multi_result = region_transfer.map_regions(proposals)
C:\Users\LuisSalazar.LY-LS-LEGION\AppData\Local\Temp\ipykernel_44344\1750804783.py:
2: UserWarning: Source region 8: no non-overlapping target assignment was available.
    multi_result = region_transfer.map_regions(proposals)
C:\Users\LuisSalazar.LY-LS-LEGION\AppData\Local\Temp\ipykernel_44344\1750804783.py:
2: UserWarning: Source region 19: approximate transfer stopped with region_limit; th
e returned region is the best candidate found, not a proven optimum.
    multi_result = region_transfer.map_regions(proposals)
C:\Users\LuisSalazar.LY-LS-LEGION\AppData\Local\Temp\ipykernel_44344\1750804783.py:
2: UserWarning: Source region 20: approximate transfer stopped with beam_limit; the
returned region is the best candidate found, not a proven optimum.
    multi_result = region_transfer.map_regions(proposals)
C:\Users\LuisSalazar.LY-LS-LEGION\AppData\Local\Temp\ipykernel_44344\1750804783.py:
2: UserWarning: Source region 20: no non-overlapping target assignment was availabl
e.
    multi_result = region_transfer.map_regions(proposals)
C:\Users\LuisSalazar.LY-LS-LEGION\AppData\Local\Temp\ipykernel_44344\1750804783.py:
2: UserWarning: Source region 25: approximate transfer stopped with beam_limit; the
returned region is the best candidate found, not a proven optimum.
    multi_result = region_transfer.map_regions(proposals)
C:\Users\LuisSalazar.LY-LS-LEGION\AppData\Local\Temp\ipykernel_44344\1750804783.py:
2: UserWarning: Source region 25: no non-overlapping target assignment was availabl
e.
    multi_result = region_transfer.map_regions(proposals)
C:\Users\LuisSalazar.LY-LS-LEGION\AppData\Local\Temp\ipykernel_44344\1750804783.py:
2: UserWarning: Source region 26: approximate transfer stopped with beam_limit; the
returned region is the best candidate found, not a proven optimum.
    multi_result = region_transfer.map_regions(proposals)
C:\Users\LuisSalazar.LY-LS-LEGION\AppData\Local\Temp\ipykernel_44344\1750804783.py:
2: UserWarning: Source region 26: no non-overlapping target assignment was availabl
e.
    multi_result = region_transfer.map_regions(proposals)
C:\Users\LuisSalazar.LY-LS-LEGION\AppData\Local\Temp\ipykernel_44344\1750804783.py:
2: UserWarning: Source region 27: approximate transfer stopped with beam_limit; the
returned region is the best candidate found, not a proven optimum.
    multi_result = region_transfer.map_regions(proposals)
C:\Users\LuisSalazar.LY-LS-LEGION\AppData\Local\Temp\ipykernel_44344\1750804783.py:
2: UserWarning: Source region 27: no non-overlapping target assignment was availabl
e.
    multi_result = region_transfer.map_regions(proposals)
C:\Users\LuisSalazar.LY-LS-LEGION\AppData\Local\Temp\ipykernel_44344\1750804783.py:
2: UserWarning: Source region 28: approximate transfer stopped with beam_limit; the
returned region is the best candidate found, not a proven optimum.
    multi_result = region_transfer.map_regions(proposals)
C:\Users\LuisSalazar.LY-LS-LEGION\AppData\Local\Temp\ipykernel_44344\1750804783.py:
2: UserWarning: Source region 28: no non-overlapping target assignment was availabl
```

```
e.
    multi_result = region_transfer.map_regions(proposals)
C:\Users\LuisSalazar.LY-LS-LEGION\AppData\Local\Temp\ipykernel_44344\1750804783.py:
2: UserWarning: Source region 29: approximate transfer stopped with beam_limit; the
returned region is the best candidate found, not a proven optimum.
    multi_result = region_transfer.map_regions(proposals)
C:\Users\LuisSalazar.LY-LS-LEGION\AppData\Local\Temp\ipykernel_44344\1750804783.py:
2: UserWarning: Source region 29: no non-overlapping target assignment was availabl
e.
    multi_result = region_transfer.map_regions(proposals)
C:\Users\LuisSalazar.LY-LS-LEGION\AppData\Local\Temp\ipykernel_44344\1750804783.py:
2: UserWarning: Source region 30: approximate transfer stopped with beam_limit; the
returned region is the best candidate found, not a proven optimum.
    multi_result = region_transfer.map_regions(proposals)
C:\Users\LuisSalazar.LY-LS-LEGION\AppData\Local\Temp\ipykernel_44344\1750804783.py:
2: UserWarning: Source region 30: no non-overlapping target assignment was availabl
e.
    multi_result = region_transfer.map_regions(proposals)
C:\Users\LuisSalazar.LY-LS-LEGION\AppData\Local\Temp\ipykernel_44344\1750804783.py:
2: UserWarning: Source region 32: approximate transfer stopped with beam_limit; the
returned region is the best candidate found, not a proven optimum.
    multi_result = region_transfer.map_regions(proposals)
C:\Users\LuisSalazar.LY-LS-LEGION\AppData\Local\Temp\ipykernel_44344\1750804783.py:
2: UserWarning: Source region 32: no non-overlapping target assignment was availabl
e.
    multi_result = region_transfer.map_regions(proposals)
C:\Users\LuisSalazar.LY-LS-LEGION\AppData\Local\Temp\ipykernel_44344\1750804783.py:
2: UserWarning: Source region 33: approximate transfer stopped with beam_limit; the
returned region is the best candidate found, not a proven optimum.
    multi_result = region_transfer.map_regions(proposals)
C:\Users\LuisSalazar.LY-LS-LEGION\AppData\Local\Temp\ipykernel_44344\1750804783.py:
2: UserWarning: Source region 34: approximate transfer stopped with beam_limit; the
returned region is the best candidate found, not a proven optimum.
    multi_result = region_transfer.map_regions(proposals)
C:\Users\LuisSalazar.LY-LS-LEGION\AppData\Local\Temp\ipykernel_44344\1750804783.py:
2: UserWarning: Source region 35: approximate transfer stopped with beam_limit; the
returned region is the best candidate found, not a proven optimum.
    multi_result = region_transfer.map_regions(proposals)
C:\Users\LuisSalazar.LY-LS-LEGION\AppData\Local\Temp\ipykernel_44344\1750804783.py:
2: UserWarning: Source region 35: no non-overlapping target assignment was availabl
e.
    multi_result = region_transfer.map_regions(proposals)
```

```

1: source=[6, 7] -> target=[29, 31]; interest=0.774
2: source=[12, 13] -> target=[28, 30]; interest=0.774
3: source=[8, 9] -> target=[27, 32]; interest=0.774
4: source=[10, 11] -> target=[26, 33]; interest=0.774
5: source=[35, 36, 37, 38] -> target=[59, 60, 61, 62]; interest=0.746
6: source=[18, 19, 20, 21, 22, 23, 24, 25, 26, 27, 28, 29, 30, 31, 32, 33] -> target
=[41, 42, 43, 44, 45, 46, 47, 48, 49, 50, 51, 52, 87, 88, 89, 90]; interest=0.730
7: source=[153, 154, 155, 156, 157, 158, 159, 160, 161, 162, 163, 164, 165, 166] ->
target=no match (not colored); interest=0.692
8: source=[62, 63, 64, 65, 66, 67, 68, 69, 70, 71, 72, 73, 74, 75] -> target=no matc
h (not colored); interest=0.692
9: source=[42, 43, 44, 45] -> target=[70, 71, 72, 73]; interest=0.690
10: source=[46, 47, 48, 49] -> target=[66, 67, 68, 69]; interest=0.690
11: source=[60] -> target=[54]; interest=0.689
12: source=[61] -> target=[53]; interest=0.689
13: source=[56] -> target=[80]; interest=0.689
14: source=[58] -> target=[35]; interest=0.689
15: source=[54] -> target=[78]; interest=0.689
16: source=[57] -> target=[36]; interest=0.689
17: source=[55] -> target=[79]; interest=0.689
18: source=[59] -> target=[34]; interest=0.689
19: source=[2, 3, 4, 5, 14, 15, 16, 17] -> target=[1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 1
1, 12, 14, 15, 16, 17, 18, 19, 20, 21, 22, 23, 24, 25]; interest=0.649
20: source=[128, 129, 130, 131, 132, 133, 134, 135, 136, 137, 138, 139, 140, 141, 14
2, 143, 144, 145] -> target=no match (not colored); interest=0.645
21: source=[50, 51, 52, 53] -> target=[74, 75, 76, 77]; interest=0.641
22: source=[39] -> target=[63]; interest=0.630
23: source=[40] -> target=[64]; interest=0.630
24: source=[41] -> target=[65]; interest=0.630
25: source=[189, 190, 191, 192, 193, 194, 195, 196, 197, 198, 199, 200, 201, 202, 20
3, 204, 205, 256] -> target=no match (not colored); interest=0.605
26: source=[222, 223, 224, 225, 226, 227, 228, 229, 230, 231, 232, 233, 234, 235, 23
6, 237, 238, 258] -> target=no match (not colored); interest=0.605
27: source=[116, 117, 118, 119, 120, 121, 122, 123, 124, 125, 126, 127] -> target=no
match (not colored); interest=0.594
28: source=[76, 77, 78, 79, 80, 81, 82, 83, 84, 85, 86, 87] -> target=no match (not
colored); interest=0.594
29: source=[167, 168, 169, 170, 171, 172, 173, 174, 175, 176, 177, 178, 179, 180, 18
1, 182, 183, 184, 185, 186, 187, 188] -> target=no match (not colored); interest=0.5
94
30: source=[206, 207, 208, 209, 210, 211, 212, 213, 214, 215, 216, 217, 218, 219, 22
0, 221, 257] -> target=no match (not colored); interest=0.585
31: source=[34] -> target=[55]; interest=0.569
32: source=[88, 89, 90, 91, 92, 93, 94, 95, 96, 97, 98, 99, 100, 101, 102, 103, 104,
105, 106, 107, 108] -> target=no match (not colored); interest=0.566
33: source=[239, 240, 241, 242, 243, 244, 245, 246, 247, 248, 249, 250, 251, 252, 25
3, 254] -> target=[112, 113, 114, 115, 116, 117, 118, 119, 120, 121, 122, 123, 124];
interest=0.557
34: source=[109, 110, 111, 112, 113, 114, 115] -> target=[106]; interest=0.550
35: source=[146, 147, 148, 149, 150, 151, 152] -> target=no match (not colored); int
erest=0.535
Colored 24 of 35 source features.

```

Symmetry Check (Optional)

Swap the parts and send every matched target region back. A robust match returns to the original source region. Look-alike copies, such as the four corner holes, may come back to another copy.

```
In [8]: reverse_transfer = RegionTransfer(target_part, source_part)
matched_regions = [region for region in multi_result.regions if region.matched]
round_trip = reverse_transfer.map_regions(Region.of(region.target_faces) for region

returned = 0
for region, back in zip(matched_regions, round_trip.regions, strict=True):
    if back.target_faces == region.source_faces:
        returned += 1
    else:
        print(
            f"source {list(region.source_faces)} -> target {list(region.target_faces)}
            f"source {list(back.target_faces)} if back.matched else 'no match'"
        )
print(f"{returned} of {len(matched_regions)} matched regions return to the same sou
```

```
source [6, 7] -> target [29, 31] -> source [10, 11]
source [12, 13] -> target [28, 30] -> source [8, 9]
source [8, 9] -> target [27, 32] -> source [12, 13]
source [10, 11] -> target [26, 33] -> source [6, 7]
source [239, 240, 241, 242, 243, 244, 245, 246, 247, 248, 249, 250, 251, 252, 253, 254] -> target [112, 113, 114, 115, 116, 117, 118, 119, 120, 121, 122, 123, 124] -> source [239, 240, 245, 251, 252, 253, 254]
source [109, 110, 111, 112, 113, 114, 115] -> target [106] -> source [135]
18 of 24 matched regions return to the same source region.
```

```
C:\Users\LuisSalazar.LY-LS-LEGION\AppData\Local\Temp\ipykernel_44344\3501231313.py:
3: UserWarning: Source region 17: approximate transfer stopped with region_limit; the
returned region is the best candidate found, not a proven optimum.
    round_trip = reverse_transfer.map_regions(Region.of(region.target_faces) for region
in matched_regions)
C:\Users\LuisSalazar.LY-LS-LEGION\AppData\Local\Temp\ipykernel_44344\3501231313.py:
3: UserWarning: Source region 23: approximate transfer stopped with region_limit; the
returned region is the best candidate found, not a proven optimum.
    round_trip = reverse_transfer.map_regions(Region.of(region.target_faces) for region
in matched_regions)
```

5. Second Case: A New Pair of Parts

Repeat the automatic mapping on a new pair: `nist_ftc_10_asme1_ct5315_rb.CATPart` as source and `nist_ftc_10_asme1_rb_ct5290_ra.CATPart` as target. Review the colored pairs rather than expecting every feature to match. The next cell opens a new pair of viewers, so the viewers above keep their colors. Run sections 1 and 2 first.

```
In [9]: cad_filename_second_source = cad_folder / "nist_ftc_10_asme1_ct5315_rb.CATPart"
cad_filename_second_target = cad_folder / "nist_ftc_10_asme1_rb_ct5290_ra.CATPart"

second_source_part = RegionPart.load_from_file(cad_filename_second_source)
second_target_part = RegionPart.load_from_file(cad_filename_second_target)
```

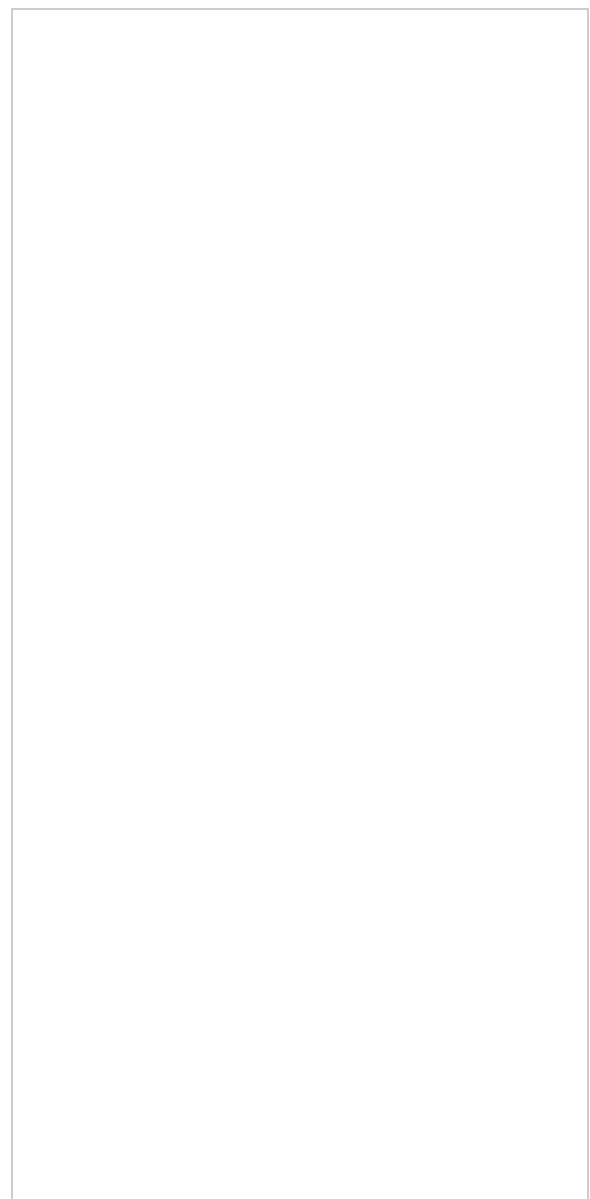
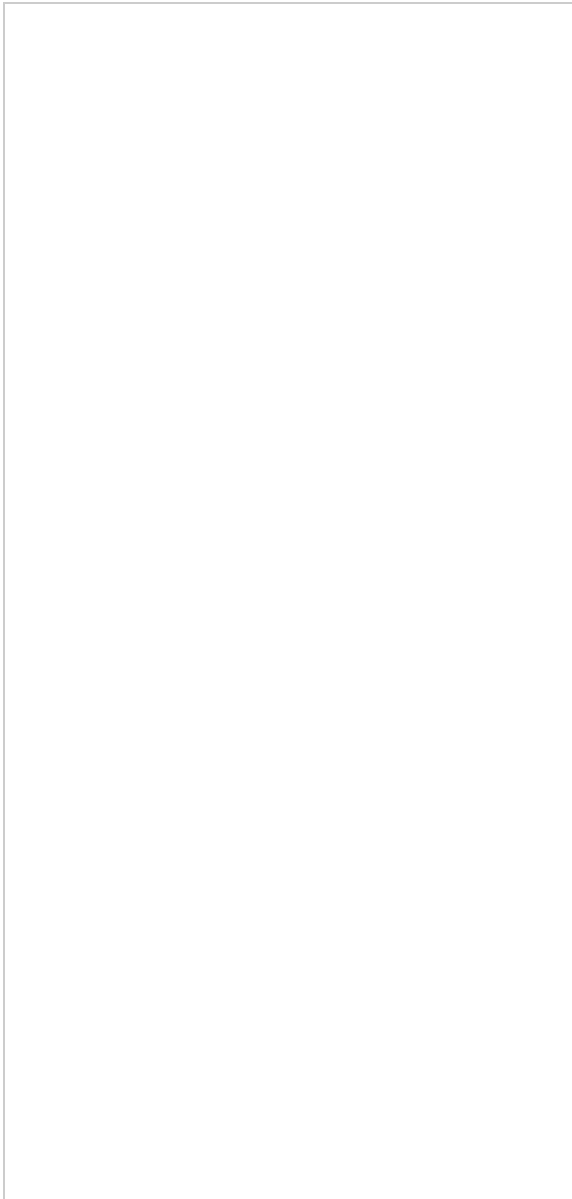
```
second_transfer = RegionTransfer(second_source_part, second_target_part)
second_proposals = second_transfer.propose_regions(second_source_part, limit=60)
```

```
In [10]: if globals().get("second_views") is not None:
        second_views.terminate()
        second_views = CompareViews.start_viewers(
            cad_filename_second_source, cad_filename_second_target, static_folder=output_dir
        )
        second_views.show()
```

Source: nist_ftc_10_asme1_ct5315_rb.CATPart

Target:

nist_ftc_10_asme1_rb_ct5290_ra.CATPart



```
In [11]: second_result = second_transfer.map_regions(second_proposals)
        second_views.highlight_regions(second_result.source_regions, second_result.target_r

        for rank, region in enumerate(second_result.regions, start=1):
            print(
                f"{rank}: source={list(region.source_faces)} -> "
```

```
        f"target={list(region.target_faces) if region.matched else 'no match (not c  
    )  
print(f"Colored {second_result.matched_count} of {len(second_proposals)} source fea
```

```
C:\Users\LuisSalazar.LY-LS-LEGION\AppData\Local\Temp\ipykernel_44344\1410563387.py:  
1: UserWarning: Source region 31: approximate transfer stopped with topology_limit;  
the returned region is the best candidate found, not a proven optimum.  
    second_result = second_transfer.map_regions(second_proposals)  
C:\Users\LuisSalazar.LY-LS-LEGION\AppData\Local\Temp\ipykernel_44344\1410563387.py:  
1: UserWarning: Source region 33: approximate transfer stopped with region_limit; th  
e returned region is the best candidate found, not a proven optimum.  
    second_result = second_transfer.map_regions(second_proposals)  
C:\Users\LuisSalazar.LY-LS-LEGION\AppData\Local\Temp\ipykernel_44344\1410563387.py:  
1: UserWarning: Source region 33: no usable target region was found under the curren  
t search settings.  
    second_result = second_transfer.map_regions(second_proposals)  
C:\Users\LuisSalazar.LY-LS-LEGION\AppData\Local\Temp\ipykernel_44344\1410563387.py:  
1: UserWarning: Source region 34: approximate transfer stopped with region_limit; th  
e returned region is the best candidate found, not a proven optimum.  
    second_result = second_transfer.map_regions(second_proposals)  
C:\Users\LuisSalazar.LY-LS-LEGION\AppData\Local\Temp\ipykernel_44344\1410563387.py:  
1: UserWarning: Source region 34: no usable target region was found under the curren  
t search settings.  
    second_result = second_transfer.map_regions(second_proposals)
```

1: source=[282, 283, 298, 299, 300, 301] -> target=[104, 105, 140, 141, 142, 143]
 2: source=[284, 285, 302, 303, 304, 305] -> target=[107, 108, 135, 136, 137, 138]
 3: source=[286, 287, 306, 307, 308, 309] -> target=[110, 111, 130, 131, 132, 133]
 4: source=[280, 281, 294, 295, 296, 297] -> target=[101, 102, 145, 146, 147, 148]
 5: source=[4, 5, 6, 8] -> target=[29, 30, 252, 253]
 6: source=[37, 38, 58, 59] -> target=[34, 223, 246, 247]
 7: source=[43, 44, 60, 61] -> target=[35, 36, 240, 241]
 8: source=[45, 46, 62, 63] -> target=[37, 38, 238, 239]
 9: source=[0, 1, 2, 3] -> target=[31, 32, 250, 251]
 10: source=[35, 36, 56, 57] -> target=[33, 224, 248, 249]
 11: source=[182, 183, 184, 185] -> target=[174, 175]
 12: source=[186, 187, 188, 189] -> target=[172, 173]
 13: source=[262, 263, 277, 278] -> target=[126, 127]
 14: source=[265, 266, 275, 276] -> target=[128, 129]
 15: source=[257, 258, 291] -> target=[120, 121, 134]
 16: source=[255, 256, 290] -> target=[122, 123, 139]
 17: source=[253, 254, 289] -> target=[124, 125, 144]
 18: source=[259, 260, 292] -> target=[118, 119, 149]
 19: source=[72, 73, 121, 122, 123, 124, 125, 126] -> target=[77, 78, 79, 80, 81, 82, 221, 222]
 20: source=[31, 48, 49, 64] -> target=[74, 75, 220, 225]
 21: source=[199, 200] -> target=[165, 166]
 22: source=[201, 202] -> target=[163, 164]
 23: source=[203, 204] -> target=[161, 162]
 24: source=[205, 206] -> target=[159, 160]
 25: source=[171, 172, 173, 174, 233, 234, 235, 236] -> target=[70, 71, 72, 73, 177, 178, 179, 180]
 26: source=[151, 152, 230] -> target=[109, 204, 205]
 27: source=[149, 150, 229] -> target=[106, 206, 207]
 28: source=[147, 148, 228] -> target=[103, 208, 209]
 29: source=[145, 146, 227] -> target=[100, 210, 211]
 30: source=[9, 10, 11, 12, 13, 14, 17, 18, 19, 20, 22, 23, 24, 26, 27, 28, 29, 30, 32, 33, 47, 51] -> target=[0, 1, 2, 3, 5, 6, 8, 9, 11, 12, 13, 14, 15, 16, 237, 256, 257, 259, 260, 261, 262, 263]
 31: source=[207, 208, 209, 210, 211, 212, 213, 214, 267, 268, 269, 270, 271, 272, 273, 274] -> target=[89, 90, 91, 92, 95, 96, 97, 98, 151, 152, 153, 154, 155, 156, 157, 158]
 32: source=[223] -> target=[197]
 33: source=[79, 87, 88, 89, 90, 91, 127] -> target=no match (not colored)
 34: source=[94, 95, 96, 97, 98, 99, 100, 101, 102, 103, 104, 105, 128] -> target=no match (not colored)
 35: source=[162, 163, 164, 165, 166] -> target=[183, 184, 185]
 36: source=[237] -> target=[113]
 37: source=[238] -> target=[114]
 38: source=[261] -> target=[117]
 39: source=[264] -> target=[116]
 40: source=[167] -> target=[182]
 41: source=[161] -> target=[186]
 Colored 39 of 41 source features.

6. Extra Example: NIST HTC, CATIA to NX

The same NIST HTC test part exported from two CAD systems: CATIA V5 as source and NX as target. Run sections 1 and 2 first.

```
In [13]: htc_catia_filename = cad_folder / "NIST-HTC-PMI-v2" / "nist_htc_V56R2023_20240530.C
htc_nx_filename = cad_folder / "NIST-HTC-NX2027_20240531.prt"

htc_catia_part = RegionPart.load_from_file(htc_catia_filename)
htc_nx_part = RegionPart.load_from_file(htc_nx_filename)
htc_transfer = RegionTransfer(htc_catia_part, htc_nx_part)
htc_result = htc_transfer.map_regions(htc_transfer.propose_regions(htc_catia_part,

for region in htc_result.regions:
    print(list(region.source_faces), "->", list(region.target_faces) if region.matc
```

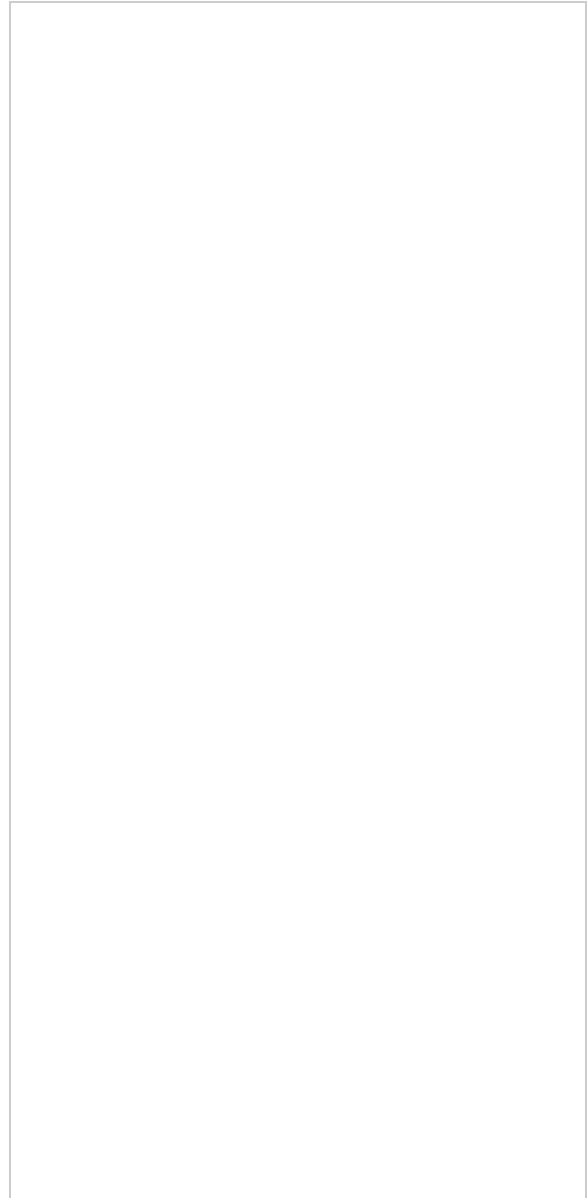
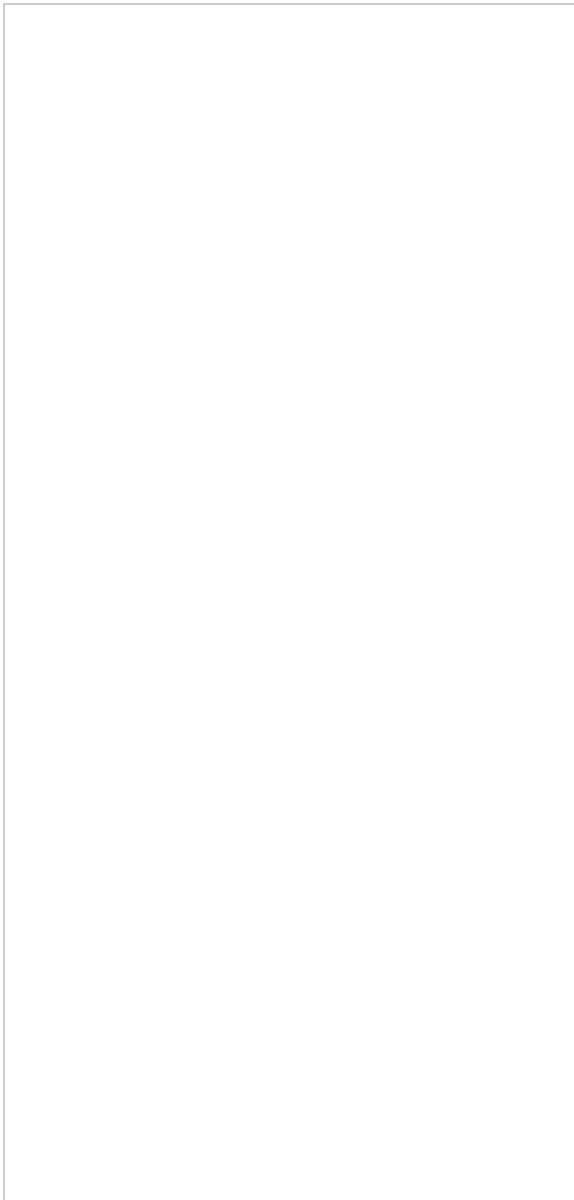
```
[56, 57, 100, 122] -> [61, 62]
[54, 55, 98, 124] -> [64, 65]
[64, 65, 102, 115] -> [67, 68]
[66, 67, 104, 113] -> [70, 71]
[68, 69, 106, 111] -> [73, 74]
[145, 146, 150, 151] -> [81, 82]
[84, 85] -> [50]
[92, 93] -> no match
[88, 89] -> no match
[86, 87] -> [46]
[90, 91] -> [48]
[116, 117, 118, 119] -> [56, 57]
[96, 128, 129] -> [53, 54]
[82, 83] -> [51]
[50, 52, 53, 94, 108, 109, 132, 133] -> [16, 17, 78, 80]
[74, 75, 76, 77, 78, 79, 135, 136, 137, 138, 139, 140, 141, 142, 143] -> [0, 1, 4,
8, 9, 10, 11, 19, 20, 21, 22, 83, 84]
[80, 81] -> [52]
[58, 59, 120] -> [58, 59]
[95, 130, 131] -> [76, 77]
[48, 49, 152] -> [13, 25, 26]
[46, 154, 155] -> [14, 15, 23]
[0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 13, 14, 15, 16, 17, 18, 19, 20] -> [100, 101,
102, 103, 104, 105, 106, 107, 108, 109, 110, 111, 112, 113, 114, 116, 117, 118, 119,
120]
[35, 36, 37, 39, 41, 42, 43, 126, 127, 156, 157, 158] -> [28, 29, 30, 31, 32, 33, 3
4, 36, 85, 86]
[62, 63, 70, 71, 72, 73, 144, 148, 149] -> [37, 38, 39, 40, 41, 42, 43, 44, 45]
[21, 22, 23, 24, 25, 26, 27, 28, 29, 30, 31, 32, 33] -> [87, 88, 89, 90, 91, 92, 93,
94, 95, 96, 97, 98, 99]
[153] -> [12]
```

```
C:\Users\LuisSalazar.LY-LS-LEGION\AppData\Local\Temp\ipykernel_44344\697398449.py:7:
UserWarning: Source region 8: no usable target region was found under the current se
arch settings.
    htc_result = htc_transfer.map_regions(htc_transfer.propose_regions(htc_catia_part,
limit=40))
C:\Users\LuisSalazar.LY-LS-LEGION\AppData\Local\Temp\ipykernel_44344\697398449.py:7:
UserWarning: Source region 9: no usable target region was found under the current se
arch settings.
    htc_result = htc_transfer.map_regions(htc_transfer.propose_regions(htc_catia_part,
limit=40))
C:\Users\LuisSalazar.LY-LS-LEGION\AppData\Local\Temp\ipykernel_44344\697398449.py:7:
UserWarning: Source region 15: approximate transfer stopped with region_limit; the r
eturned region is the best candidate found, not a proven optimum.
    htc_result = htc_transfer.map_regions(htc_transfer.propose_regions(htc_catia_part,
limit=40))
```

```
In [14]: if globals().get("htc_views") is not None:
        htc_views.terminate()
        htc_views = CompareViews.start_viewers(htc_catia_filename, htc_nx_filename, static_
        htc_views.show()
```

Source:
nist_htc_V56R2023_20240530.CATPart

Target: NIST_HTC_NX2027_20240531.prt



```
In [15]: htc_views.highlight_regions(htc_result.source_regions, htc_result.target_regions)
```

```
Out[15]: [(242, 12, 12),
          (73, 122, 242),
          (157, 242, 36),
          (191, 10, 169),
          (57, 191, 169),
          (191, 117, 29),
          (40, 7, 140),
          None,
          None,
          (46, 140, 42),
          (140, 21, 61),
          (12, 156, 242),
          (228, 242, 73),
          (199, 36, 242),
          (10, 191, 100),
          (191, 85, 57),
          (29, 43, 191),
          (57, 140, 7),
          (140, 42, 108),
          (21, 136, 140),
          (242, 184, 12),
          (150, 73, 242),
          (36, 242, 70),
          (191, 10, 33),
          (57, 114, 191),
          (144, 191, 29)]
```

```
In [ ]:
```